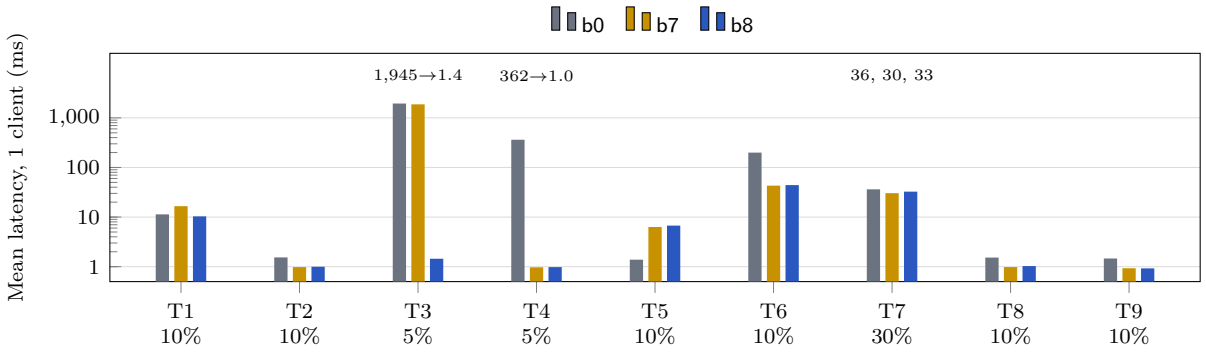


Figure 1: Mean latency of each transaction at one client, before (b0) and after applying workload A's design (b7, b8); the second label line is the transaction's weight in workload B. T3 and T4 fall by three orders of magnitude once they reach a single shard, while T7, 30% of the mix, stays at about 30 ms.

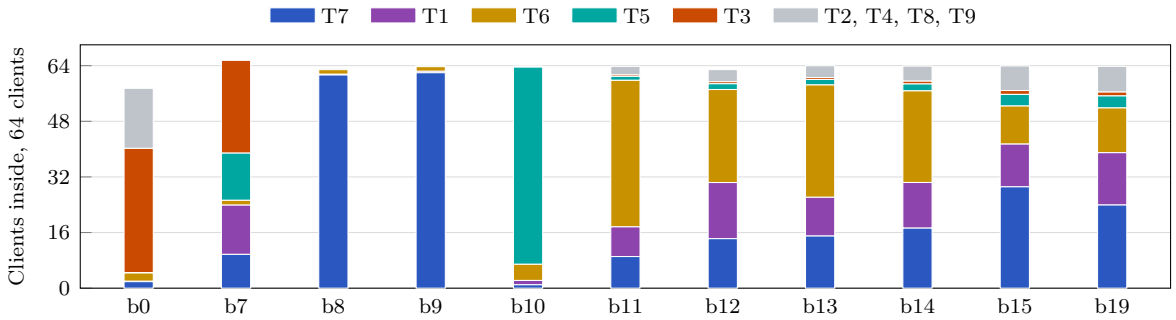


Figure 2: Where the 64 clients are at any moment, by Little's law (throughput × mean latency of each transaction), along the main chain. Each fix moves the queue to the next transaction: T3 (b0, b7), T7 (b8, b9), T5 (b10), T6 (b11–b14) and T7 again, now without a majority (b15 on). Grey is mostly T4 in b0 (17.2 clients). Bars below 64 (b0) or above it (b7) come from transactions of several seconds that straddle the end of the 600 s run.

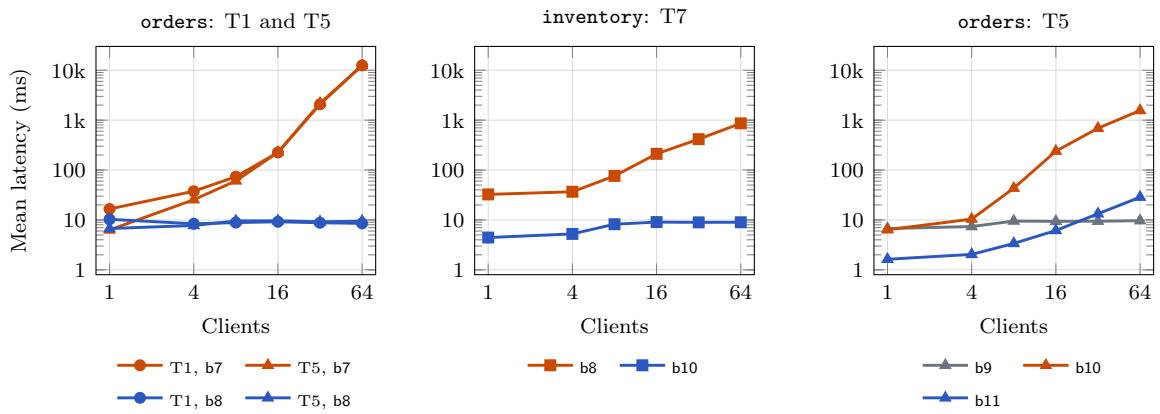


Figure 3: Three times in the chain, a transaction whose `UPDATE` lacks `merchant_id` goes to every shard and Citus lets only one such multi-shard modification hold a shard at a time, so these transactions queue behind each other (orange); routing them to one shard removes the queue (blue). Left: in b7, T1's final `UPDATE orders` and T5 share one queue and have the same latency at every client count (12,538 and 12,545 ms at 64). Middle: T7's updates by `inventory_id` in b8 complete 64.8–71.2 times per second from 4 clients on, whatever the load. Right: once T7 is routed (b10), T5, unchanged since b8, hits the same kind of limit at 35.5–36.3 per second from 8 clients on.

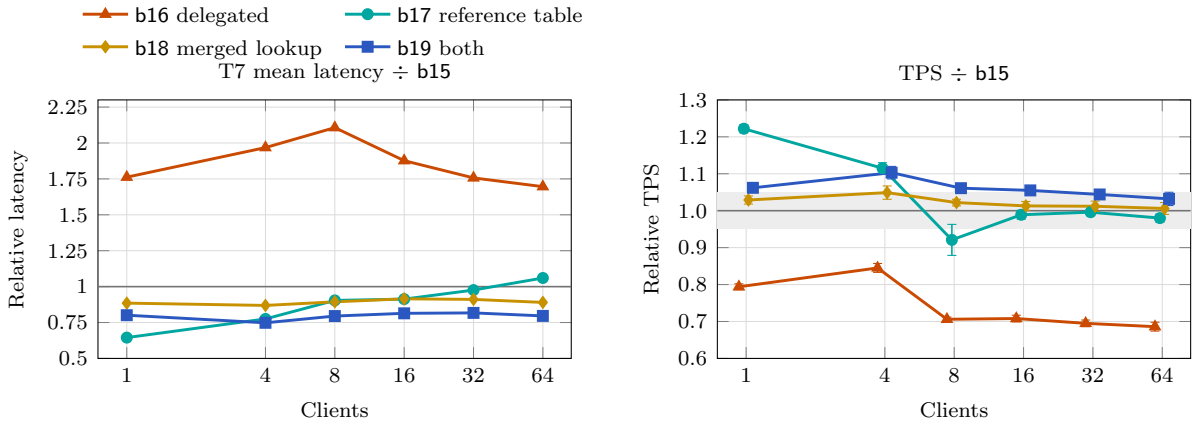


Figure 4: The four ways of routing T7 that branch from b15, at every client count. Left: T7's own mean latency. Right: total throughput; error bars are 95% intervals from ten 60 s windows of the same run (within-run noise only), and the shaded band is the ±5% run-to-run variation of the conditions check. Cutting T7's latency by a fifth (b19) raises TPS by 6–10% at 1–8 clients but only 3% at 64.

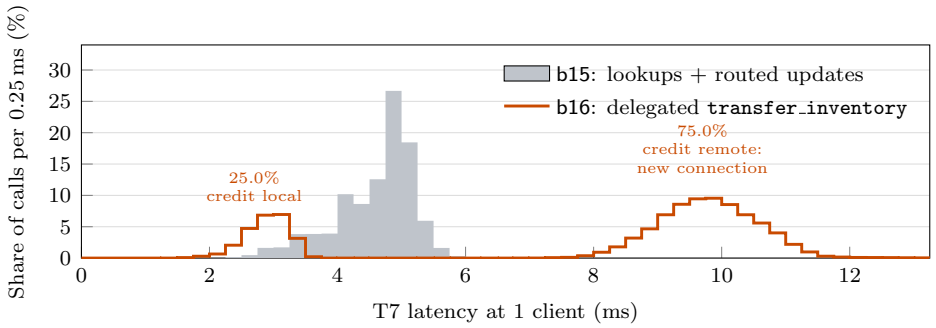


Figure 5: Distribution of T7's latency at one client. Delegated to the source merchant's worker (b16), T7 is bimodal: 25.0% of calls take about 2.5 ms, the rest about 9.5 ms, with no call in between. The fast share matches the probability that both products' merchants sit on the same worker, 25.03%, computed from Citus's hash of `merchant_id`; a remote credit costs about 7 ms because the worker backend opens a new connection on every call.

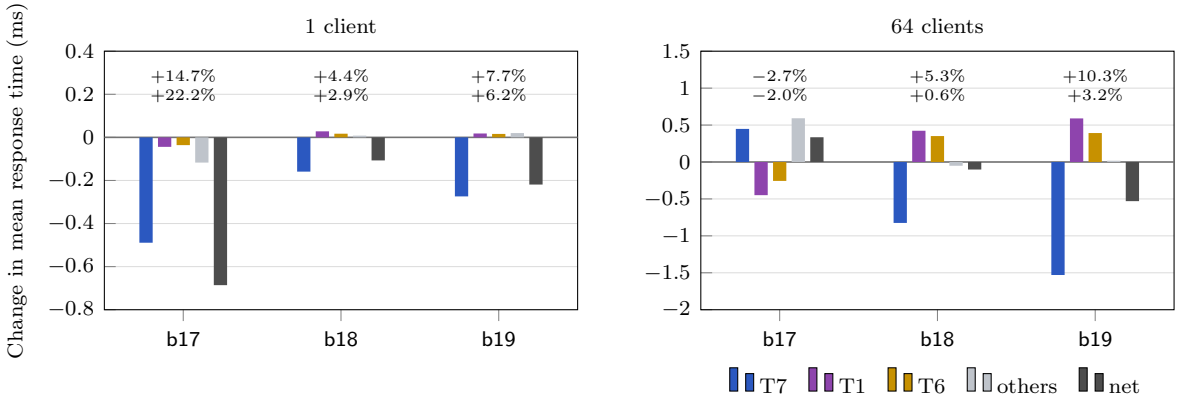


Figure 6: Where T7's saving went. Each bar is a transaction's weight times the change in its mean latency against b15; they sum to the change in mean response time per transaction (net), and with no think time TPS = clients / response time, which predicts every measured ratio here within 0.2%. The labels give the TPS change that T7's latency alone would predict (top) and the measured change (bottom). At 64 clients T1 and T6 slow down by 0.98 ms of the 1.53 ms that T7 saves in b19, so the predicted +10% becomes +3%. b16 is left out: its T7 adds 5.2 ms at 64 clients.